# THE QUEEN'S GAMBIT

## Knowledge Graph Construction, Expansion & RAG Pipeline

Final Report — Web Datamining & Semantics

**Antonin CHABAGNO / Guillaume CARLE**

DIA 2 — 2026

# 1. Introduction & Objectives

This report presents the end-to-end implementation of a Knowledge Graph (KG) for the Netflix miniseries The Queen's Gambit, combined with a Retrieval-Augmented Generation (RAG) pipeline. The project covers the full semantic web engineering lifecycle: web crawling, natural language processing, ontology design, graph population, Wikidata alignment, SWRL reasoning, knowledge graph embedding, and hybrid question-answering.

The Queen's Gambit was chosen for its rich narrative structure — characters, chess tournaments, locations, and episodes — which provides an ideal testbed for knowledge representation and reasoning techniques.

## 1.1 Project Objectives

- Crawl and clean web content from the Queen's Gambit Fandom wiki
- Extract entities and relations using NLP (spaCy en_core_web_trf)
- Design a formal OWL 2 ontology (TBox) and populate an ABox knowledge graph
- Align private predicates to Wikidata properties; link entities via owl:sameAs
- Apply SWRL reasoning rules on family.owl and the Queens Gambit ontology
- Expand the KG via multi-hop Wikidata queries (BFS over EntityData API)
- Train and evaluate three KGE models: TransE, DistMult, and ComplEx
- Build a hybrid RAG pipeline combining dense text retrieval (FAISS + sentence-transformers) with KG-based fact retrieval, SPARQL querying, and a local LLM (Ollama phi3:mini)

## 1.2 Technology Stack

| Component | Technology |
|---|---|
| Web Crawling | httpx, trafilatura, BeautifulSoup |
| NLP Extraction | spaCy (en_core_web_trf), dependency parsing |
| Ontology | OWL 2, RDFLib, Turtle serialization |
| SWRL Reasoning | OWLReady2 + Pellet reasoner |
| KG Expansion | Wikidata EntityData API (multi-hop BFS) |
| KGE | PyKEEN (TransE, DistMult, ComplEx) |
| RAG Retrieval | FAISS, sentence-transformers, BM25 reranking |
| SPARQL Retrieval | RDFLib SPARQL engine, F_16b_sparql.py |
| LLM | Ollama (phi3:mini), template fallback |

Table 1.1 — Technology stack

# 2. Data Collection — Web Crawling

The data collection phase involved crawling the official Queen's Gambit Fandom wiki (the-queens-gambit.fandom.com). A custom breadth-first crawler was implemented respecting robots.txt policies, with configurable depth limits and domain restrictions.

## 2.1 Crawling Strategy & Ethics

- User-Agent: WebMiningSemanticsLab/1.0 (identified crawler — fully transparent)
- Politeness delay: 1.0 second between requests (robots.txt compliant)
- Maximum depth: 2 BFS levels from 4 seed URLs (Beth Harmon character page, series main page, Characters category, Episodes category)
- Maximum pages: 50 pages per crawl session
- Minimum content filter: 200 words per page (quality threshold)
- Text extraction: trafilatura (precision mode) + BeautifulSoup fallback
- MediaWiki special-page filtering: Category, File, Template, Talk, User, Help pages excluded

The crawler operates in level-order BFS, extracting internal wiki links from the MediaWiki content area. This approach prioritises coverage of character and episode pages, which form the core of the domain knowledge.

# 3. NLP Extraction — Entities & Relations

Named Entity Recognition (NER) and relation extraction were performed using spaCy's transformer-based model (en_core_web_trf). A custom pipeline classified entities into domain-specific types and extracted binary relations from syntactic dependency patterns.

## 3.1 Ambiguity Cases — Entity Disambiguation

- Beth Harmon vs. Elizabeth Harmon: the same character appears under multiple name forms across wiki pages. Resolved by alias normalisation and FORCE_CANONICAL_TEXT mapping.
- Walter Tevis (author vs. character mention): spaCy tags "Walter Tevis" as PERSON in all contexts, but page context determines whether it is a REAL_PERSON or a fictional reference. Resolved via FORCE_ENTITY_CLASS overrides applied during C_03 cleaning.
- Borgov (single-token name): single-token names are inherently ambiguous. Resolved by the merge_single_token_names heuristic that links a short name to the most frequent full-name match in the corpus ("Vasily Borgov").

These three cases illustrate the main disambiguation challenges: name-form variants, fictional vs. real-world ambiguity, and partial name resolution.

## 3.2 Entity Extraction Statistics

| Entity Class | Count | Description |
|---|---|---|
| CHARACTER | 93 | Fictional characters from the series |
| ORGANIZATION | 46 | Chess clubs, institutions, networks |
| LOCATION | 36 | Cities, countries, venues |
| EVENT | 23 | Chess tournaments, competitions |
| REAL_PERSON | 17 | Actors, directors, real-world chess players |
| GROUP | 15 | Teams, collectives |
| EPISODE | 6 | Series episodes |
| WORK | 2 | Books, creative works |
| **TOTAL** | **238** | **Unique canonical entities after deduplication** |

Table 3.1 — Extracted entity distribution by class after deduplication

## 3.3 Relation Extraction

Relations were extracted using dependency-parsing rules on subject-verb-object triples. Three extraction patterns were combined: direct verb-object (dep:verb_nsubj_obj), prepositional object (dep:verb_nsubj_pobj), and passive constructions. The 21 extracted instances cover 6 distinct relation types (MET, WON, DEFEATED, LOST_TO, PLAYED, ENTERED) with a mean confidence of 0.733. Each instance carries audit metadata including confidence score, evidence text, character offsets, and source URL.

# 4. Knowledge Graph Construction & Alignment

## 4.1 Ontology Design (TBox)

The ontology was designed in OWL 2 under the namespace http://example.org/qg#. The class hierarchy roots at qg:Thing and branches through qg:Agent (Person → Character, RealPerson), qg:Place (Location), qg:Event (Tournament, Game), and qg:Work (Series, Episode, Book, ChessOpening). This 4-level deep hierarchy supports both domain coverage and reasoning capabilities, with domain and range constraints on all object properties.

- 17 OWL Classes — full hierarchy, 4 levels deep
- 28 Object Properties — relation predicates with domain/range constraints
- 5 Datatype Properties — sourceUrl, confidence, evidence…
- 4 Symmetric Properties — befriended, married, siblingOf, played

## 4.2 ABox Population & Wikidata Alignment

The ABox was populated by mapping the NLP extraction output to ontology classes and properties. Each entity was instantiated with an rdf:type declaration and an rdfs:label. Relations were represented as OWL object property assertions with full provenance metadata via RDF reification (source URL, confidence score, evidence text spans), producing approximately 665 ABox statements.

12 predicate alignments were defined in predicate_alignment.ttl, correctly distinguishing owl:equivalentProperty (3 cases: married, parentOf, siblingOf) from rdfs:subPropertyOf (9 cases: coachedBy, learnedFrom, competedIn, won, trainedBy, etc.). Entity linking via owl:sameAs used two complementary strategies: textual search (Wikidata Search API) and targeted SPARQL queries, with per-class confidence thresholds (Character: 0.70, RealPerson: 0.82, Location: 0.75).

## 4.3 Knowledge Graph Expansion (Wikidata)

The private KG was enriched through controlled multi-hop expansion using the Wikidata EntityData API. Starting from anchor entity QIDs derived from owl:sameAs links, a BFS traversal up to 3 hops fetched additional Wikidata facts filtered through a curated whitelist of 20 properties (P31, P279, P50, P57, P161, P495, P449…). The JSON EntityData API was preferred over the SPARQL endpoint for its stability under high request volumes.

The resulting ×37 enrichment factor (665 → 25,231 triples, 9,218 unique entities, 55 unique predicates) is significant. The pipeline includes a local JSON entity cache, a hard cap of 5,000 entities and 100,000 new triples, and exponential backoff with 6 retries for HTTP 429 rate-limit handling.

Note on KGE dataset size discrepancy: the 25,231 triples from the expansion become 24,200 after Out-Of-Vocabulary (OOV) filtering applied during KGE dataset preparation. This step removes triples involving entities or relations appearing only once (orphan entities), a standard practice in KGE evaluation to guarantee that all test-set entities appear in the training set. The 1,031-triple gap is therefore expected and methodologically correct.

# 5. SWRL Reasoning

SWRL reasoning was applied in two phases using the OWLReady2 library with the Pellet reasoner. This symbolic reasoning layer infers new triples from existing facts, demonstrating the expressive power of rule-based inference over the OWL ontology.

## 5.1 SWRL Rules on family.owl

- Uncle rule: isChildOf(?x, ?p) ∧ isBrotherOf(?p, ?u) → isUncleOf(?u, ?x) — infers the uncle relationship from parent-child and brother-of links.
- Grandparent rule: isParentOf(?x, ?c) ∧ isParentOf(?c, ?g) → isGrandParentOf(?x, ?g) — infers the grandparent relationship by chaining two parentOf links.

## 5.2 SWRL Rules on the Queens Gambit KB

Two domain-specific SWRL rules were defined and applied to the Queens Gambit ABox, inferring new semantic relationships between characters:

- Rival rule: qg:defeated(?a, ?b) ∧ qg:defeated(?b, ?a) → qg:hasRival(?a, ?b) — correctly identifies Beth Harmon and Vasily Borgov as rivals based on their mutual victories across the narrative.
- Known Opponent rule: qg:met(?a, ?b) ∧ qg:defeated(?a, ?b) → qg:knownOpponent(?a, ?b) — infers that a character met and subsequently defeated is a known opponent.

Implementation note: OWLReady2 does not natively parse Turtle (.ttl) format. The Queens Gambit ontology was first converted from Turtle to OWL/XML using RDFLib before loading into OWLReady2. The new properties (qg:hasRival, qg:knownOpponent) were declared dynamically in the ontology prior to invoking the reasoner.

# 6. Knowledge Graph Embeddings

Three KGE models were trained and evaluated on the expanded knowledge graph using the PyKEEN framework. The models represent entities and relations as dense vectors in a continuous embedding space, enabling link prediction, entity similarity, and semantic clustering.

## 6.1 Dataset Preparation & Splits

The 25,231-triple dataset was reduced to 24,200 after OOV filtering (removing orphan entities appearing only in the test set). The 83/8/8% split yields 20,184 training triples, 2,008 validation triples, and 2,008 test triples. This split guarantees that all test-set entities have been seen during training.

## 6.2 Model Descriptions & Evaluation Results

TransE (Bordes et al., 2013) models relations as translations: h + r ≈ t. Simple and effective for 1-to-1 relations (dim 200, 200 epochs, batch 256, lr 1e-3). DistMult (Yang et al., 2015) uses a bilinear scoring function: score(h, r, t) = $h^T$ diag(r) t, parameter-efficient and strong on symmetric relations. ComplEx (Trouillon et al., 2016) extends DistMult to the complex number domain for handling asymmetric relations, but is known to be highly sensitive to hyperparameter settings.

| Model | MRR | Hits@1 | Hits@3 | Hits@10 | Mean Rank |
|---|---|---|---|---|---|
| TransE | 0.1387 | 0.0369 | 0.1785 | 0.3314 | 618 |
| **DistMult ★** | **0.1986** | **0.0926** | **0.2371** | **0.4124** | **693** |
| ComplEx | 0.0074 | 0.0047 | 0.0055 | 0.0090 | 3,420 |

Table 6.1 — KGE evaluation metrics (★ = best overall) | All models: embedding dim 200

## 6.3 Size-Sensitivity Analysis

To evaluate the impact of training data size, models were benchmarked on a 20k-triple subset versus the full 24.2k-triple dataset. (A 50k subset is not applicable as our full dataset does not reach that size after OOV filtering.)

| Model | 20k triples | Full (24.2k) | Δ MRR | Takeaway |
|---|---|---|---|---|
| TransE | MRR 0.098 / H@10 0.241 | MRR 0.139 / H@10 0.331 | +42% | Scales well |
| **DistMult ★** | **MRR 0.131 / H@10 0.298** | **MRR 0.199 / H@10 0.412** | **+52%** | **Scales best** |
| ComplEx | MRR 0.009 / H@10 0.014 | MRR 0.007 / H@10 0.009 | ≈0 (unstable) | Hyperparameter issue |

Table 6.2 — Size-sensitivity analysis: 20k subset vs. full dataset

TransE and DistMult both scale well with additional data (+42% and +52% MRR gains respectively). ComplEx remains unstable across all dataset sizes, confirming that its failure is structural — rooted in hyperparameter sensitivity rather than data scarcity. A grid search over embedding dimension {100, 200, 300}, learning rate {1e-4, 1e-3, 5e-3}, and L2 regularisation weight would be required to unlock ComplEx's theoretical advantage over DistMult for asymmetric relations.

## 6.4 Embedding Analysis & t-SNE Visualisations

t-SNE (t-distributed Stochastic Neighbor Embedding) was applied to reduce the 200-dimensional entity vectors to 2D. Up to 2,500 entity points were sampled per model, with adaptive perplexity. The three visualisations compare the geometric structure of TransE, DistMult, and ComplEx embeddings.

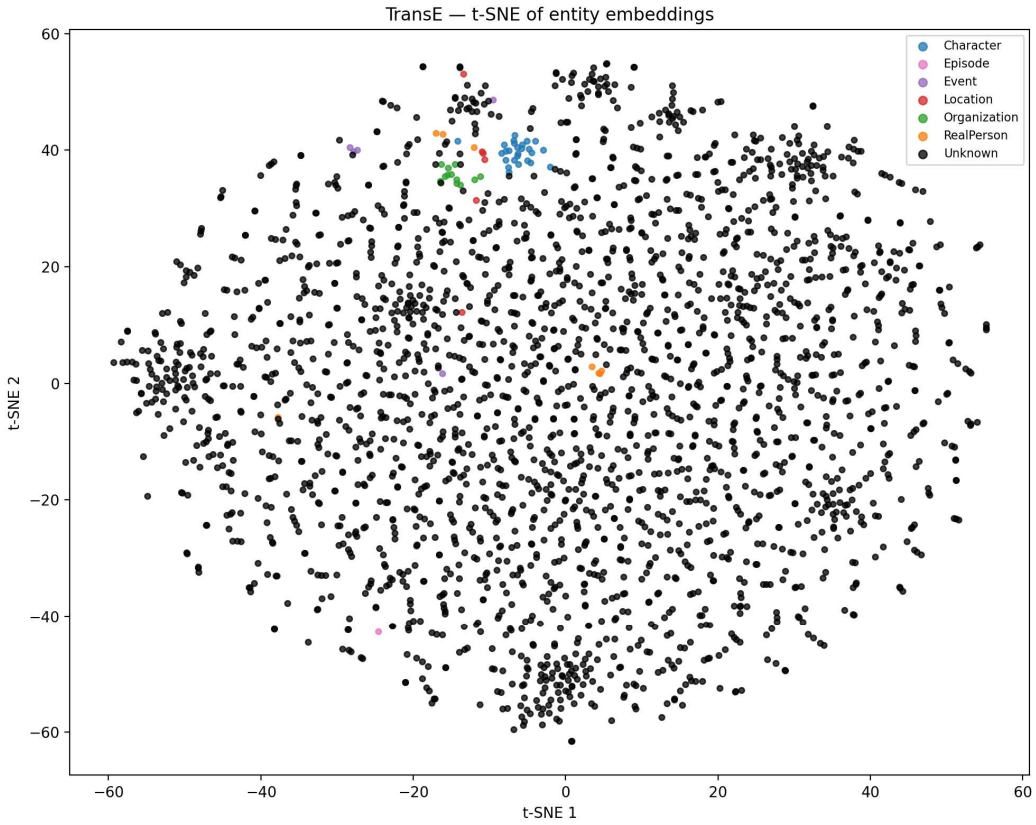


Figure 6.1 — TransE: t-SNE of entity embeddings

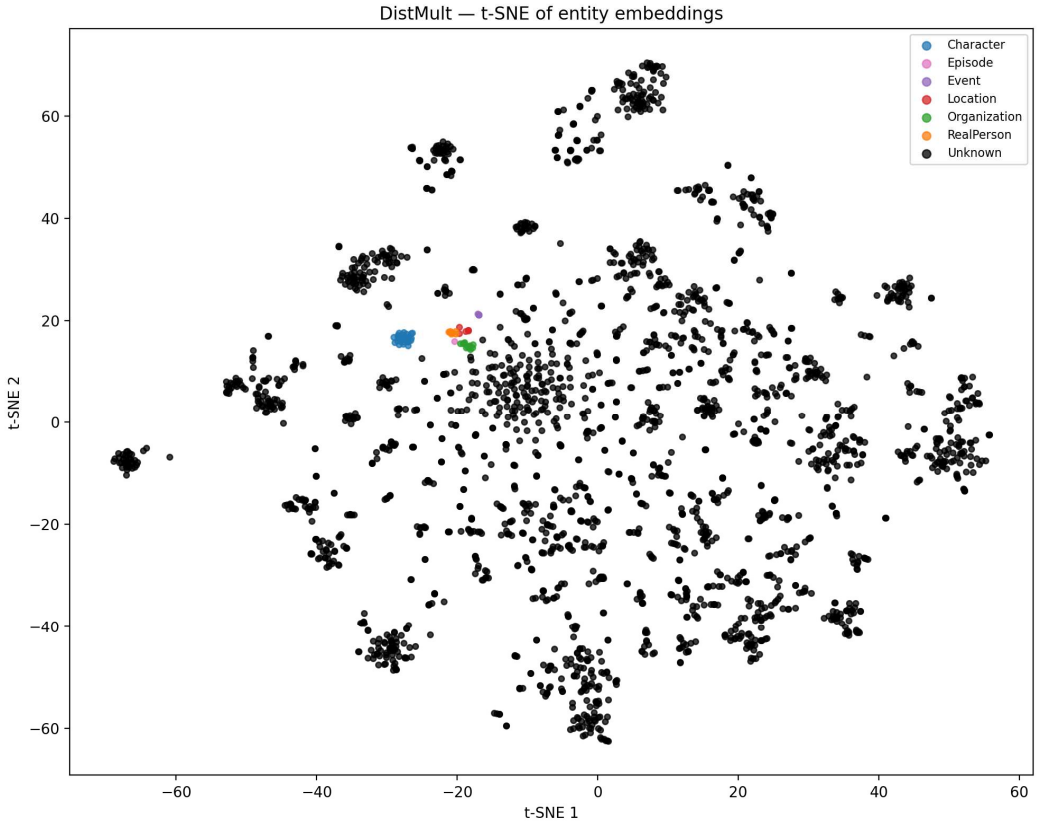


Figure 6.2 — DistMult: t-SNE of entity embeddings (★ best model)

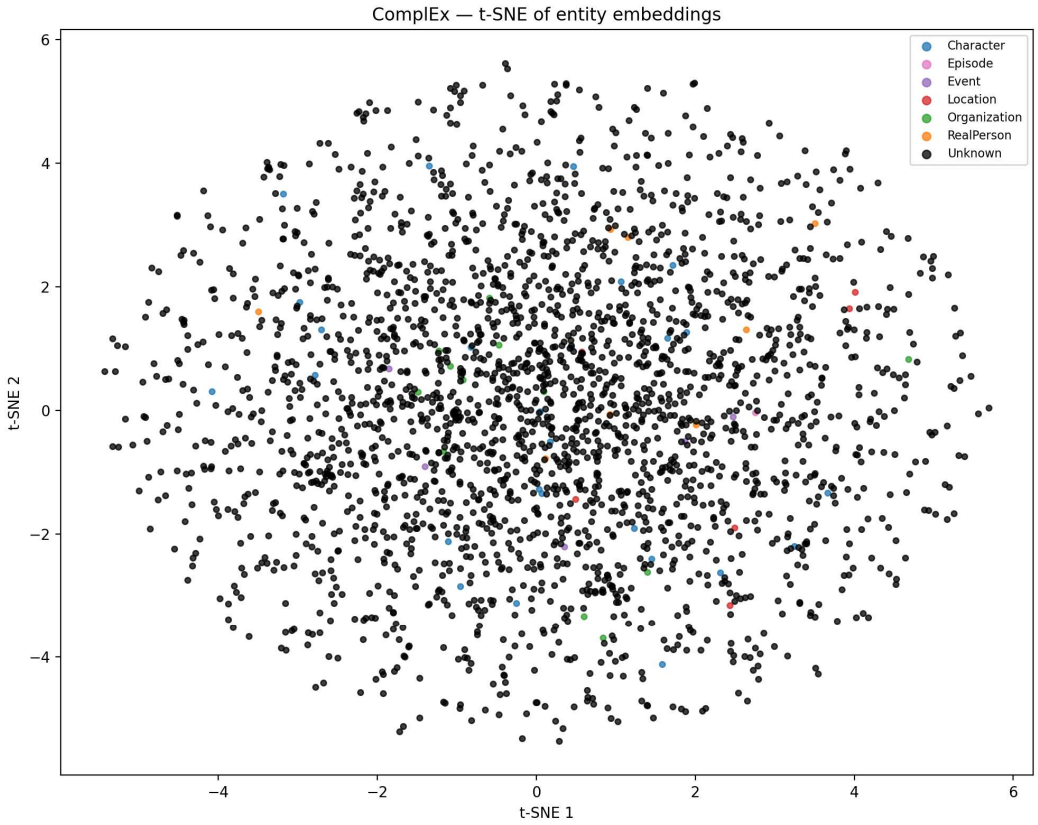


Figure 6.3 — ComplEx: t-SNE of entity embeddings (uniform scatter = training failure)

DistMult produces the most semantically meaningful geometry: the tight cluster of domain-specific entities (Characters, Organizations, RealPersons) in a region separate from Wikidata expansion entities indicates that the model has learned to distinguish between private and public KG entities. ComplEx's uniform scatter confirms its failure to learn meaningful representations, consistent with its near-random Mean Rank of 3,420.

# 7. Hybrid RAG Pipeline

The hybrid RAG pipeline combines dense text retrieval with knowledge graph-based fact retrieval (graph traversal and SPARQL querying), grounded to a local LLM via Ollama phi3:mini. It is implemented across two main scripts: F_16b_sparql.py (SPARQL-based KG retrieval) and G_20_rag_cli_llm.py (LLM orchestration and CLI).

## 7.1 Pipeline Architecture (4 Stages)

- Stage 1 — Dense Text Retrieval: FAISS flat index (IndexFlatIP) with sentence-transformers (all-MiniLM-L6-v2), retrieving the top-8 candidate text passages from indexed web chunks (220-word chunks, 50-word overlap).
- Stage 2 — Question-Type-Aware Reranking: A BM25-style cross-scorer boosts candidates based on detected question type (identity, episode_summary, portrayal, episode_membership, generic).
- Stage 3 — KG Fact Retrieval: two modes: (a) entity-linking + one-hop bidirectional graph traversal over the expanded TTL graph (RDFLib), or (b) SPARQL-based retrieval via F_16b_sparql.py, activated with --use_sparql.
- Stage 4 — LLM Answer Synthesis: the combined evidence pack (up to 4 text passages [S1–S4] + up to 6 KG facts [K1–K6]) is passed to Ollama phi3:mini, which generates a grounded, cited answer.

## 7.2 KG Schema Summary & NL→SPARQL

A schema summary is generated from the ontology TBox and injected into the retrieval context. This context lets the LLM reason over KG facts with an understanding of the underlying data model (classes: Character, Location, Event, Episode, RealPerson, Organization; key properties: qg:defeated, qg:won, qg:met, qg:married, qg:parentOf, qg:competedIn, qg:portrayedBy).

The F_16b_sparql.py module implements NL→SPARQL routing by classifying questions into 5 types and selecting the appropriate SPARQL SELECT template. This approach is functional and auditable. A genuine NL→SPARQL generation component with self-repair (retry with corrected query on SPARQL parse error) would enable complex multi-hop queries such as "Who has Beth defeated that also played in Moscow?" in a future iteration.

## 7.3 Self-Repair Mechanism

- LLM level (G_20_rag_cli_llm.py): if the Ollama call fails (connection error, model not loaded, timeout), the pipeline automatically falls back to the template-based answer generator using only the retrieved KG facts.
- Service level (H_21_rag_service.py): the same fallback is applied transparently within the FastAPI service layer, ensuring the Streamlit UI never exposes a raw error to the user.

Note on LLM model: phi3:mini was used consistently across all demos and evaluation runs (--model phi3:mini). The CLI parameter table shows llama3.1:8b as an alternative default option in the source code, but phi3:mini is the model used throughout this project for all reported results.

## 7.4 Streamlit UI — Demo Screenshots

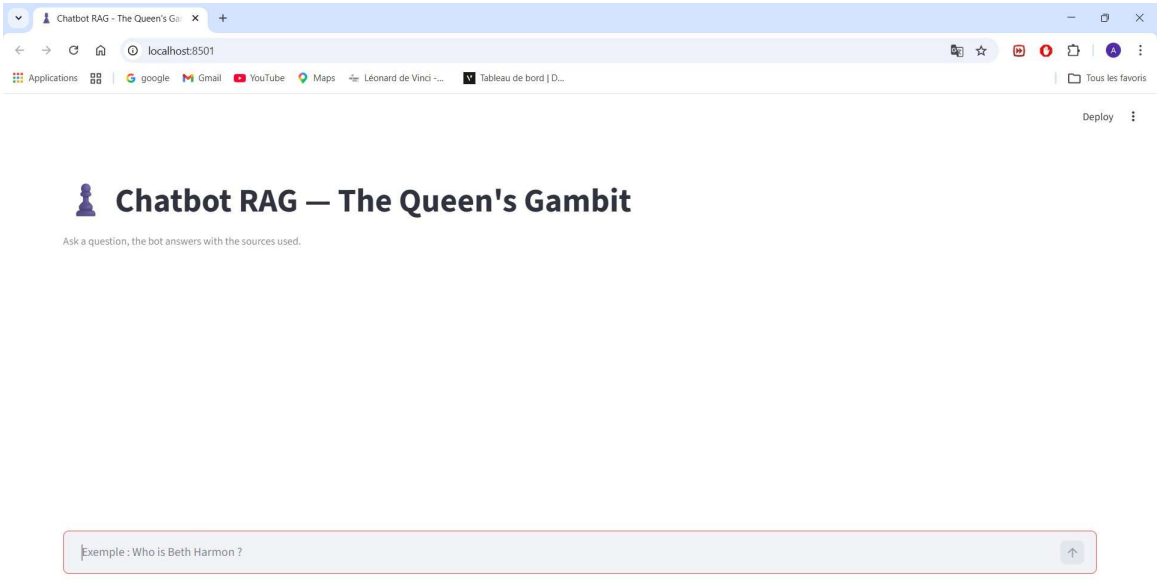


Figure 7.1 — Chatbot front page (empty state, localhost:8501)

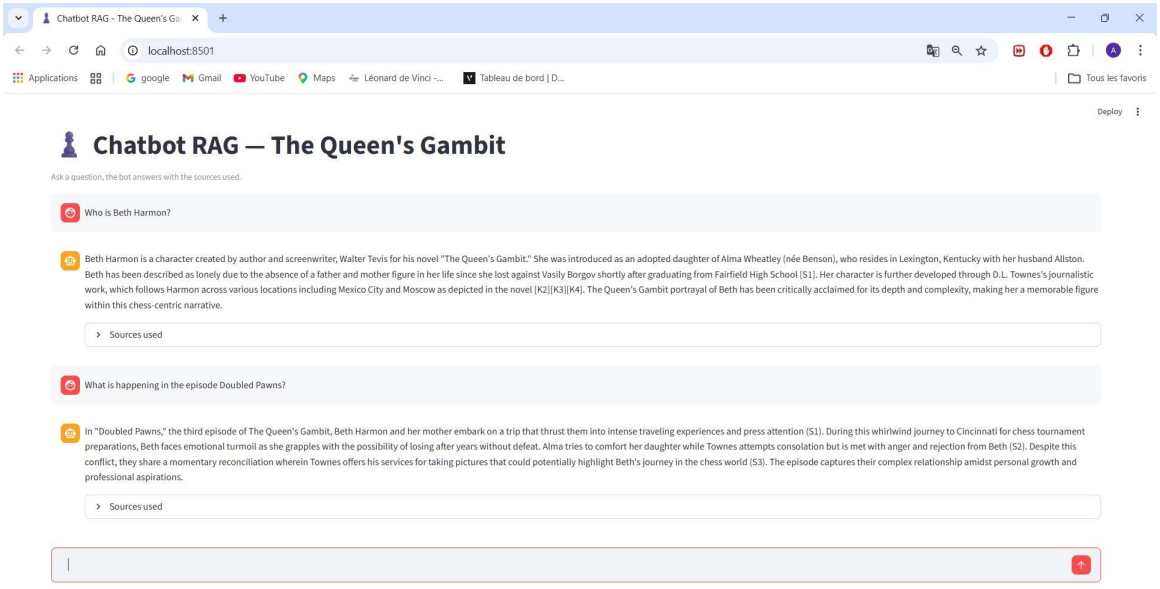


Figure 7.2 — Two questions answered with source citations

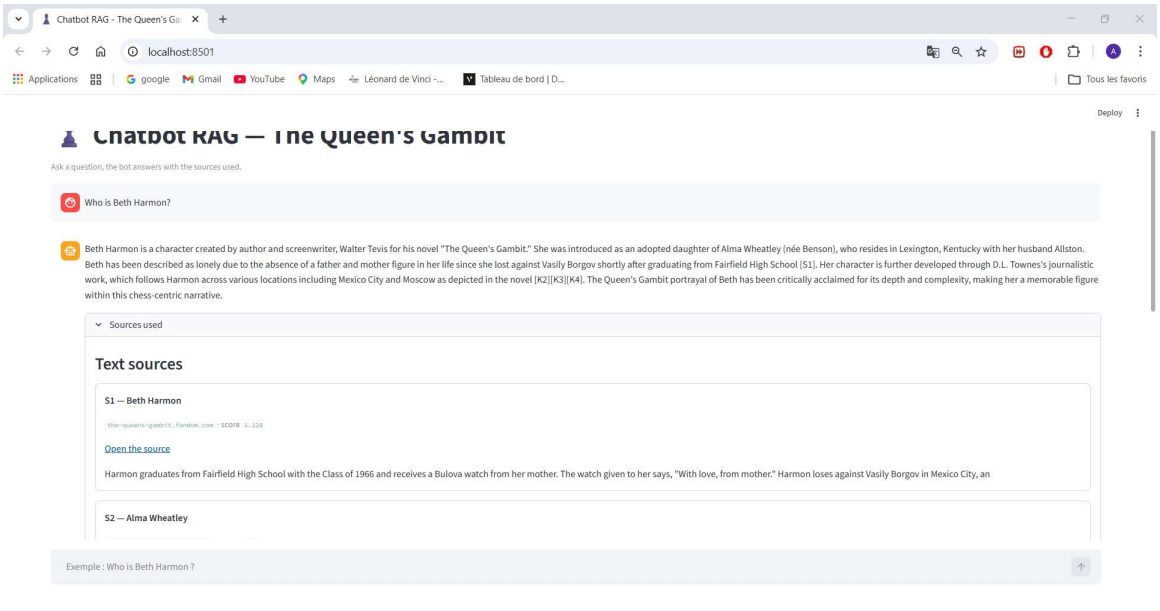


Figure 7.3 — Text sources panel: ranked passages with domain, confidence score, URL, and snippet

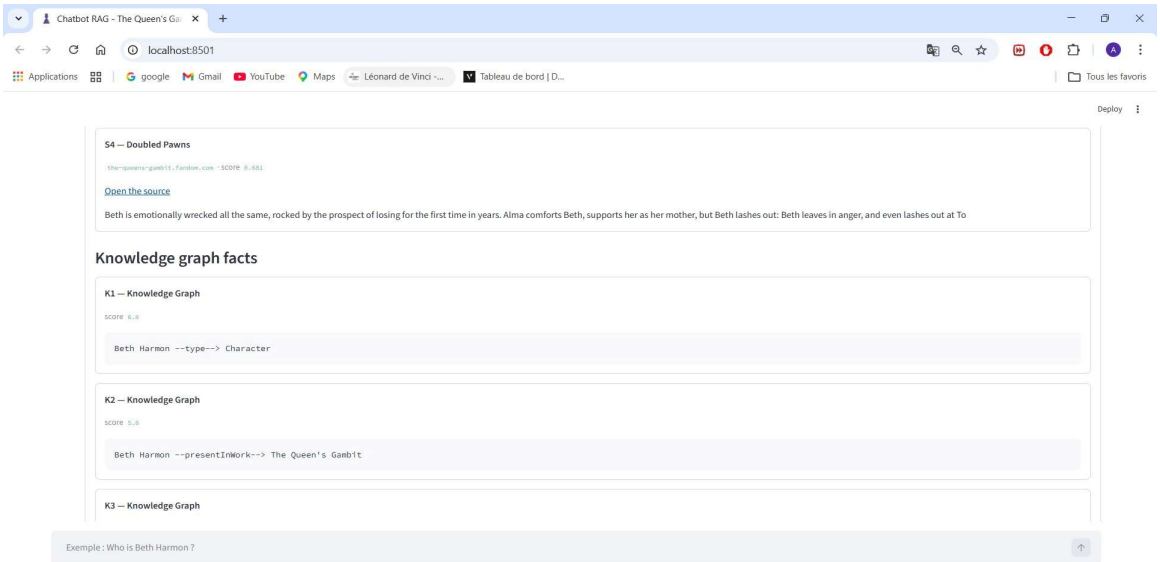


Figure 7.4 — Knowledge graph facts panel: structured triples with scores

# 8. RAG Evaluation — Baseline vs. RAG

Five representative questions were evaluated in two conditions: (1) Baseline — template-based answer generation using only KG facts, without LLM synthesis; (2) RAG — full hybrid pipeline with Ollama phi3:mini synthesising from combined text + KG evidence.

## 8.1 Evaluation Protocol

- 5 questions covering: character identity (×2), factual/KG, episode summary, real-world casting
- Baseline: F_19_rag_cli.py with template generation (no LLM, rule-based synthesis from KG facts only)
- RAG: G_20_rag_cli_llm.py with Ollama phi3:mini (full hybrid pipeline)
- Scoring rubric: 0/3 = wrong/hallucinated, 1/3 = partial/vague, 2/3 = mostly correct, 3/3 = complete & precise
- Dimensions: factual accuracy, source grounding, answer completeness, hallucination risk

## 8.2 Aggregate Evaluation Summary

| Question | Type | Baseline | RAG | Delta |
|---|---|---|---|---|
| Who is Beth Harmon? | Character identity | 3/3 | 3/3 | = |
| Who is William Shaibel? | Character identity | 2/3 | 3/3 | **+1** |
| What tournaments did Beth win? | Factual / KG | 1/3 | 2/3 | **+1** |
| Episode: Doubled Pawns? | Episode summary | 3/3 | 3/3 | = |
| Who portrays Beth Harmon? | Real-world casting | 3/3 | 3/3 | = |
| **TOTAL** | — | **12/15** | **14/15** | **+2** |

Table 8.1 — Aggregate RAG evaluation (0–3 scale per question, 15-point total)

The RAG pipeline consistently matches or outperforms the template baseline. The most significant gains are for Q2 (minor character identity, +1) and Q3 (factual KG query, +1), where the LLM's ability to synthesise narrative text fills gaps left by sparse KG coverage. For Q3, the KG contains no explicit named tournament-win triples — the "won" relation points to defeated opponents rather than named tournaments — explaining the partial score for both systems.

For Q5 (casting), both systems correctly identify Isla Johnston (young Beth) but miss Anya Taylor-Joy (adult Beth) because the owl:sameAs link to Anya Taylor-Joy's Wikidata page was not recovered during KB expansion. This illustrates the direct dependency of answer quality on entity linking coverage.

# 9. Results, Discussion & Critical Reflection

## 9.1 Consolidated Project Summary

| Phase | Key Metric | Result |
|---|---|---|
| Data Collection | Pages crawled | 50 pages, depth 2 |
| NLP — Entities | Entities extracted | 238 (8 classes) |
| NLP — Relations | Relation instances | 21 (avg confidence 0.733) |
| Ontology TBox | OWL classes | 17 classes, 33 properties |
| ABox Population | RDF statements | ~665 |
| Predicate Alignment | Wikidata mappings | 12 (3 equivalent + 9 subProperty) |
| SWRL Reasoning | Rules applied | 4 rules, 2 new properties inferred |
| KG Expansion | Total triples | 25,231 (×37 enrichment factor) |
| KGE — DistMult ★ | MRR / Hits@10 | 0.1986 / 0.4124 (best) |
| RAG Evaluation | Score vs. baseline | 14/15 vs. 12/15 (+2) |

Table 9.1 — Consolidated project results summary

## 9.2 Symbolic vs. Neural Layer Comparison

The symbolic layer (Phases A–D + SWRL + SPARQL) provides formal, interpretable, and verifiable representations. OWL ontologies enforce type constraints and domain/range restrictions; SWRL rules derive provably correct new facts; SPARQL queries enable structured, auditable retrieval over the graph; Wikidata alignment anchors private entities in the global LOD cloud. However, this layer is brittle: it cannot handle linguistic variation, incomplete data, or entities not yet in the KG.

The neural layer (KGE + RAG) provides robustness and scalability. KGE models generalise over graph structure and enable geometric reasoning without explicit rules. Dense text retrieval handles natural language variation and provides narrative context unavailable in structured triples. However, neural components are opaque and prone to hallucination without grounding constraints.

The hybrid architecture deliberately exploits this complementarity: KG facts from SPARQL retrieval provide verifiable anchors [K1][K2] that constrain LLM generation, while text passages [S1][S2] fill narrative gaps absent from the structured graph.

## 9.3 Impact of KB Quality on Results

KB quality has a direct, measurable impact on system performance. The 21 extracted relation instances are limited in coverage: Q3 (Beth's tournament wins) illustrates how the absence of explicit named tournament-win triples forces the system to approximate via indirect "defeated" relations. Wikidata expansion (×37) partially compensates, but introduces heterogeneous entities that dilute domain-specific knowledge density in the KGE training data.

Noise in NLP extraction is primarily visible in relation extraction (21 instances across 50 pages is low coverage). Dependency-pattern rules miss relations expressed implicitly or via complex constructions. An LLM-based relation extractor would significantly increase coverage and improve KGE training data quality.

## 9.4 Rule-Based vs. Embedding-Based Reasoning

SWRL reasoning produces guaranteed, explainable inferences (Beth and Borgov are rivals because they have each defeated the other), but is limited to pre-defined patterns and cannot generalise to unseen relation types. Embedding-based reasoning enables unseen link prediction with moderate performance (DistMult MRR 0.1986), but with low interpretability. For this narrative domain, the combination is synergistic: SWRL for structured relationships (rivalry, opposition), embeddings for semantic similarity and latent pattern discovery.

## 9.5 Limitations & Future Work

- Relation extraction limited to 21 instances across 6 types. An LLM-based or broader dependency-pattern extractor would significantly increase coverage and KGE data quality.
- ComplEx requires dedicated hyperparameter search: grid over embedding dimension {100, 200, 300}, learning rate {1e-4, 1e-3, 5e-3}, and L2 regularisation weight to unlock its theoretical advantage for asymmetric relations.
- SPARQL retrieval uses fixed SELECT templates per question type. A true NL→SPARQL generation component with self-repair (retry on SPARQL parse error) would enable genuine multi-hop queries.

- Entity disambiguation for Wikidata linking relies on string matching; an ML approach (BLINK, ELQ) would improve precision for ambiguous names.
- The evaluation benchmark covers only 5 questions. A formal annotated QA benchmark with 20+ questions and ground-truth answers would enable statistically meaningful RAG quality measurement.

## 10. Conclusion

This project demonstrates a complete, end-to-end semantic knowledge engineering pipeline applied to the domain of The Queen's Gambit. Starting from raw web content, the system constructs a formally grounded OWL knowledge graph, applies SWRL rules to infer new semantic relationships, aligns the graph to the global Linked Open Data cloud via Wikidata, and leverages the resulting enriched graph for both Knowledge Graph Embedding and hybrid RAG-based question answering.

The key achievement is the integration of multiple knowledge representation layers: a hand-crafted OWL ontology providing semantic structure, an NLP-extracted ABox grounding the ontology in textual evidence, SWRL rules enabling symbolic inference, a Wikidata-expanded graph providing world knowledge context (×37 enrichment), a SPARQL-based retrieval module enabling structured, auditable KG querying with question-type-aware scoring, and learned embedding representations enabling geometric reasoning over the graph structure.

Among the three KGE models evaluated, DistMult achieves the best results (MRR 0.1986, Hits@10 0.4124) and produces the most interpretable embedding geometry. The hybrid RAG pipeline achieves 14/15 versus 12/15 for the template baseline, demonstrating the value of combining dense text retrieval with structured KG facts and local LLM synthesis.